

Algorithms

Lecture 8

Shortest Path Algos (Dijkstra, Bellman-Ford)

Prantik Paul [PNP]

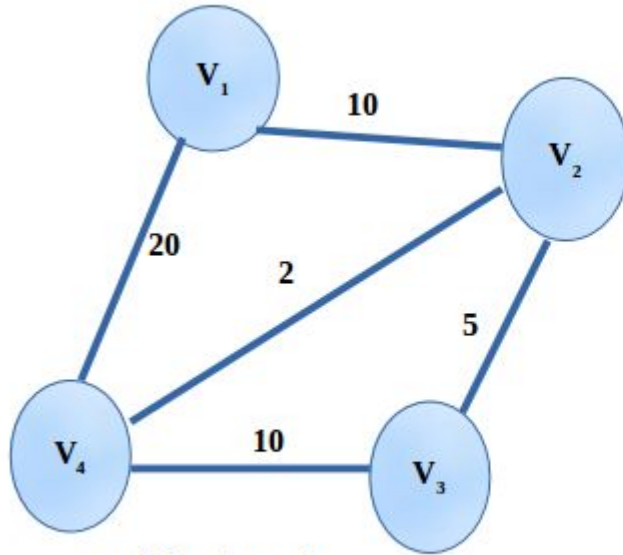
Lecturer

Department of Computer Science and Engineering

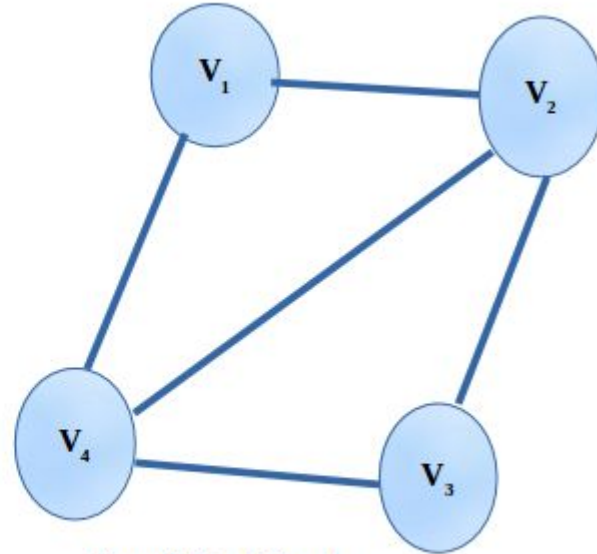
BRAC University

Adapted From: Slides by Dr. Md Abul Kashem Mia, CSE, BUET

Weighted Graph

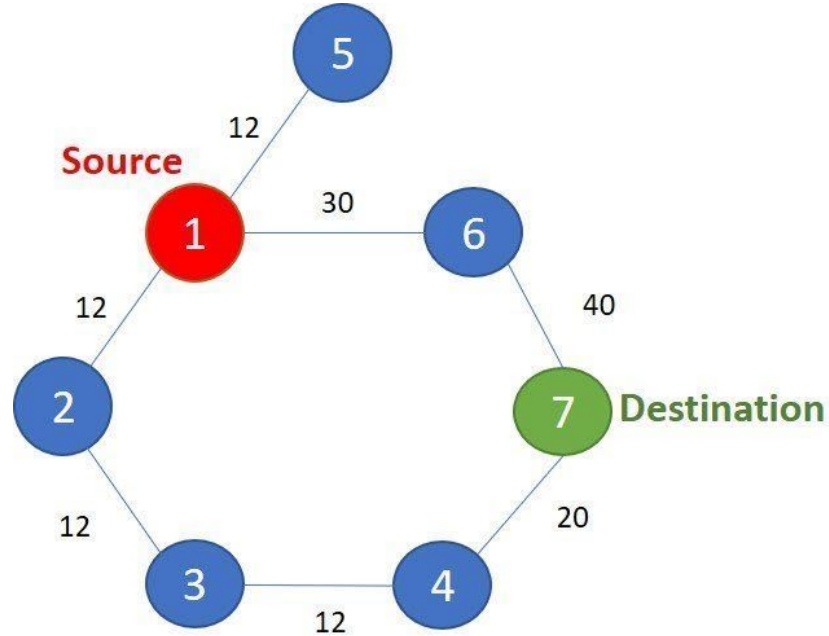


Weighted Graph



Unweighted Graph

Weighted Graph and Shortest Path



Weighted Shortest Path:

1,2,3,4,7

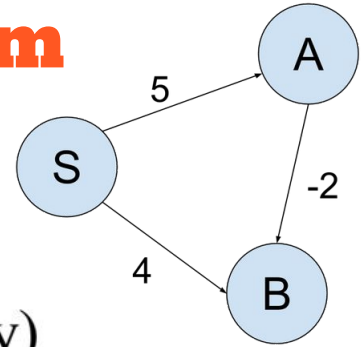
Unweighted Shortest Path:

1,6,7

Shortest Path Problem

- Shortest path = a path of the minimum weight
- Shortest-Path problems
 - **Single-Source:** Find a shortest path from a given source (vertex s) to each of the vertices.
 - **Single-Destination:** Find a shortest path to a given destination (vertex t) from each of the vertices.
 - **Single-Pair:** Given two vertices, find a shortest path between them. Solution to single-source problem solves this problem efficiently, too.
 - **All-Pairs:** Find shortest-paths for every pair of vertices. Dynamic programming algorithm.

Single-Source Shortest Path Problem

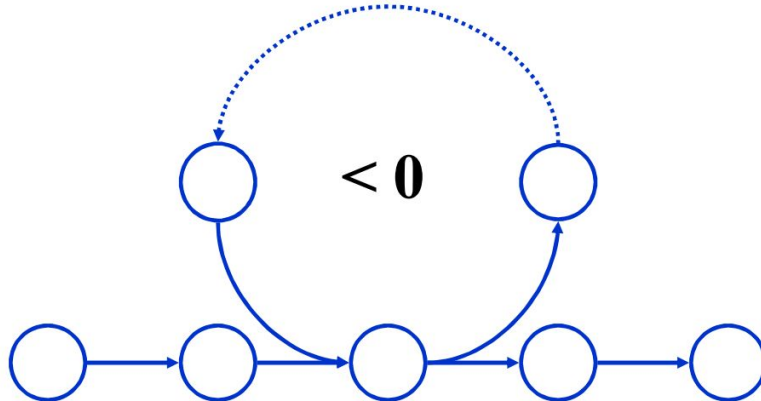


- We will discuss two general case algorithms:
 - **Dijkstra's Algorithm** (positive edge weights only)
 - **Bellman-Ford Algorithm** (positive and negative edge weights)
- If all edge weights are equal (let's say 1), the problem is solved by BFS in $\Theta(V + E)$ time.

Negative Weight Cycle

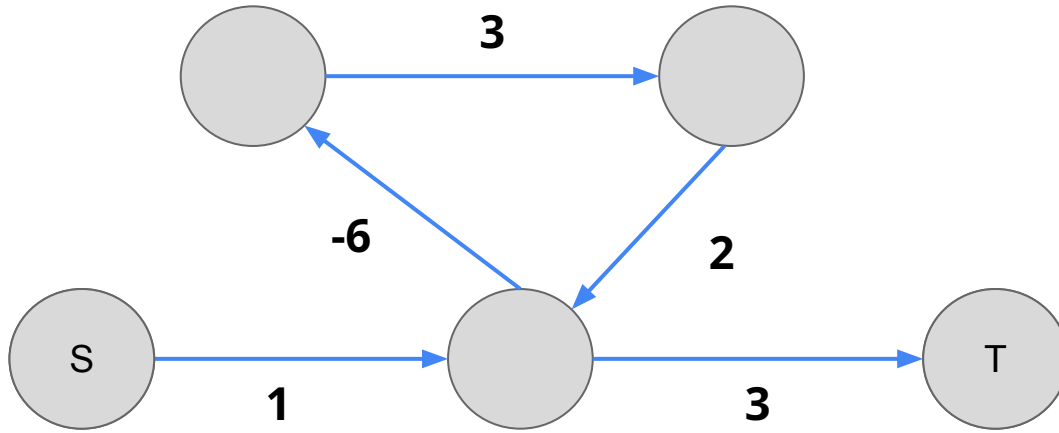
If there is a cycle for which the total weight is negative, then Bellman Ford cannot find the shortest path.

- In graphs with negative weight cycles, some shortest paths will not exist (*Why ?*):



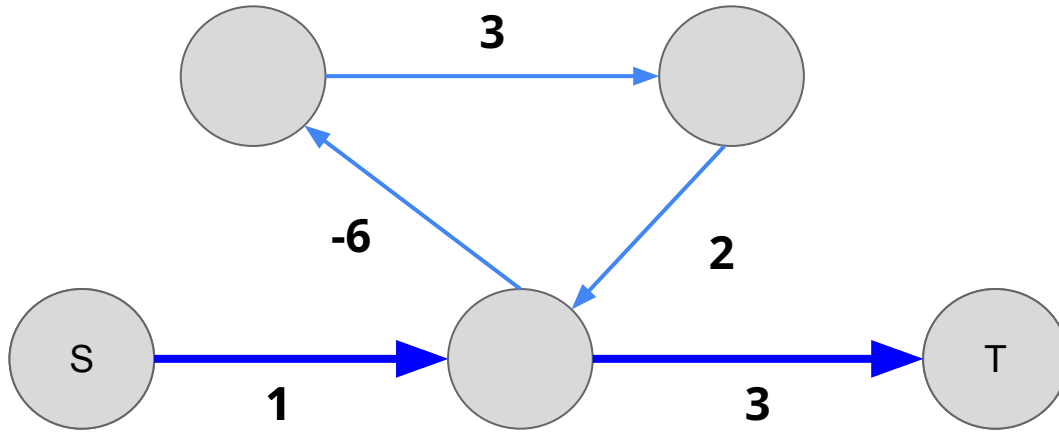
Negative Weight Cycle

- Suppose the graph has vertices like these



Negative Weight Cycle

- Suppose the graph has more vertices like these

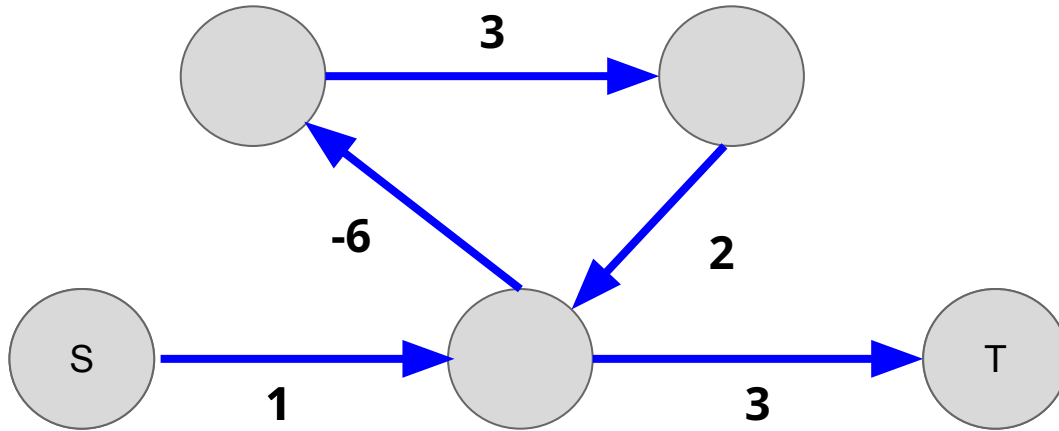


- Distance = $1+3=4$

path	distance
1	4

Negative Weight Cycle

- Suppose the graph has more vertices like these

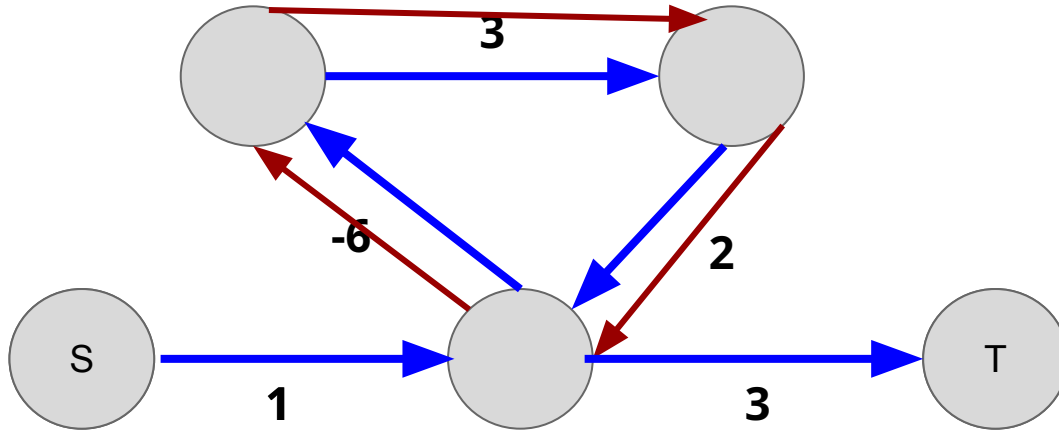


- Distance = $1 - 6 + 3 + 2 + 3 = 3$

path	distance
1	4
2	3

Negative Weight Cycle

- Suppose the graph has more vertices like these



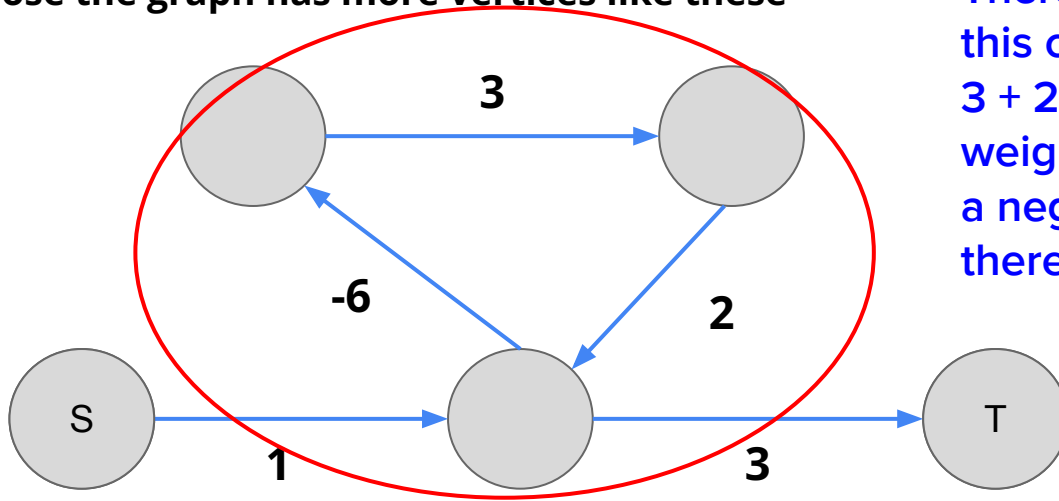
path	distance
1	4
2	3
3	2

- Distance = $1 - 6 + 3 + 2 - 6 + 3 + 2 + 3 = 2$

Will keep going reducing the distance, therefore shortest path cannot be found

Negative Weight Cycle

- Suppose the graph has more vertices like these



There is a cycle here and for this cycle the total weight is $3 + 2 - 6 = -1$. Since the total weight is negative, it will form a negative weight cycle and there will be no shortest path.

Dijkstra's Algorithm

- If no negative edge weights, we can beat Bellman-Ford Algorithm
- Similar to breadth-first search
 - Grow a tree gradually, advancing from vertices taken from a queue

Relaxation

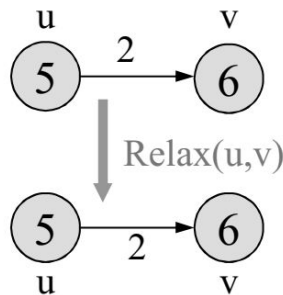
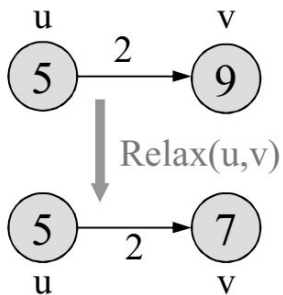
- A key technique in shortest path algorithms is *relaxation*
 - **Idea**: for all v , maintain upper bound $d[v]$ on $\delta(s, v)$

```
Relax(u, v, w) {  
    if (d[v] > d[u] + w(u, v))  
        then d[v] = d[u] + w(u, v);  
}
```

<Distance
from
source>

$$5 + 2 = 7 < 9$$

Update
distance
of v to 7 from 9



$$5 + 2 = 7 > 6$$

So, do not
update

Dijkstra Algorithm Pseudocode

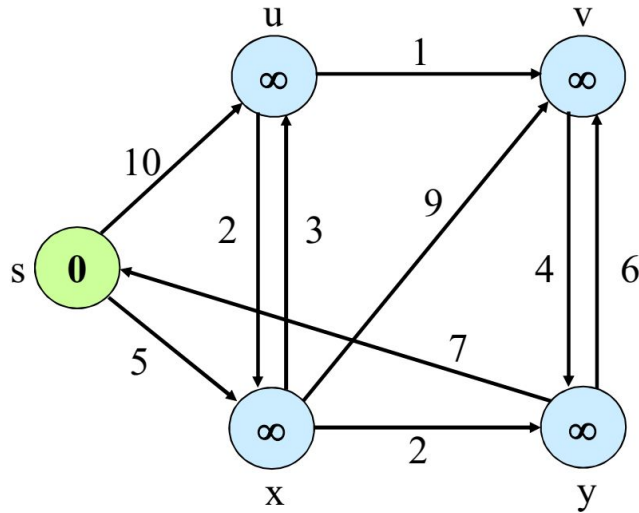
```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
14      if dist[u] + w(u, v) < dist[v]:
15        dist[v] ← dist[u] + w(u, v)
16        prev[v] ← u
17
18
19  return dist[], prev[]
```

Relaxation

Very similar to BFS

- Instead of regular Queue, we use Min Heap

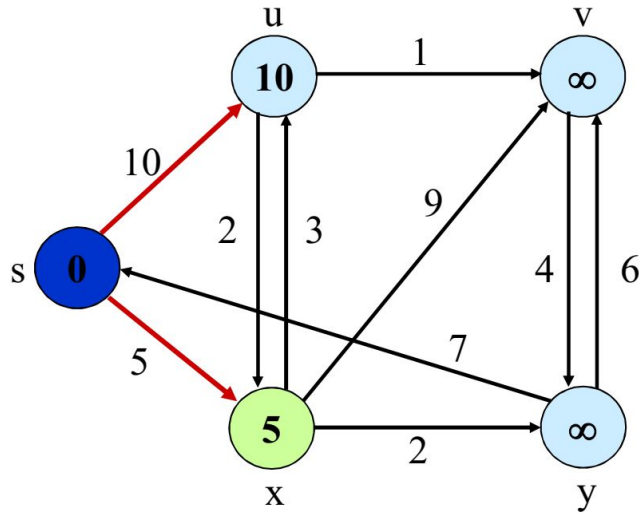
Dijkstra's Algorithm: Example



Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
14      if dist[u] + w(u, v) < dist[v]:
15        dist[v] ← dist[u] + w(u, v)
16        prev[v] ← u
17
18  return dist[], prev[]
```

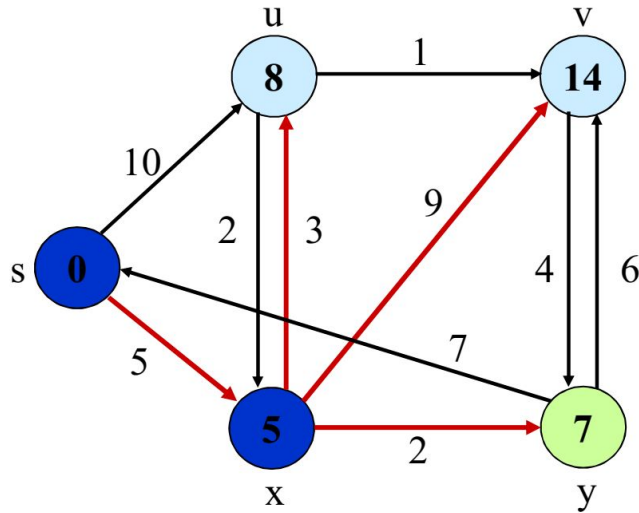
Dijkstra's Algorithm: Example



Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
14      if dist[u] + w(u, v) < dist[v]:
15        dist[v] ← dist[u] + w(u, v)
16        prev[v] ← u
17
18  return dist[], prev[]
```

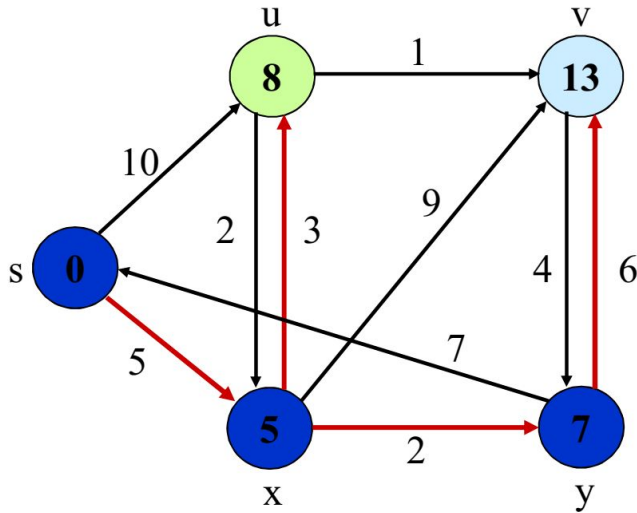
Dijkstra's Algorithm: Example



Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
14      if dist[u] + w(u, v) < dist[v]:
15        dist[v] ← dist[u] + w(u, v)
16        prev[v] ← u
17
18  return dist[], prev[]
```

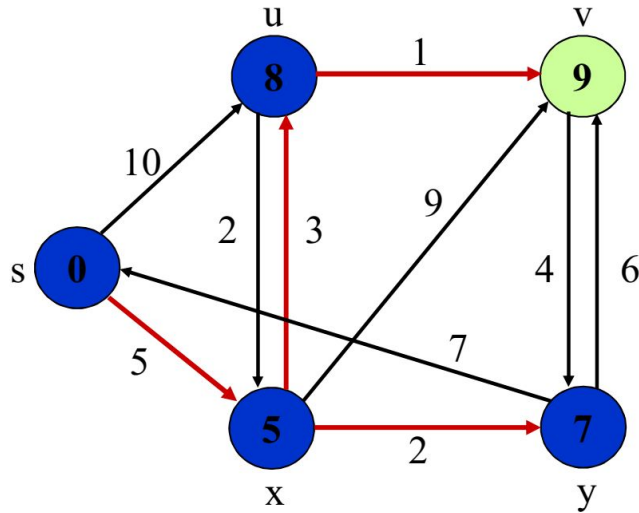
Dijkstra's Algorithm: Example



Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
14      if dist[u] + w(u, v) < dist[v]:
15        dist[v] ← dist[u] + w(u, v)
16        prev[v] ← u
17
18  return dist[], prev[]
```

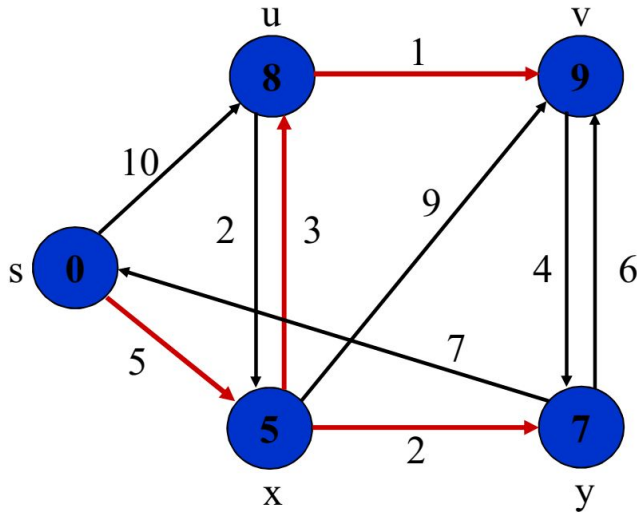
Dijkstra's Algorithm: Example



Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
14      if dist[u] + w(u, v) < dist[v]:
15        dist[v] ← dist[u] + w(u, v)
16        prev[v] ← u
17
18  return dist[], prev[]
```

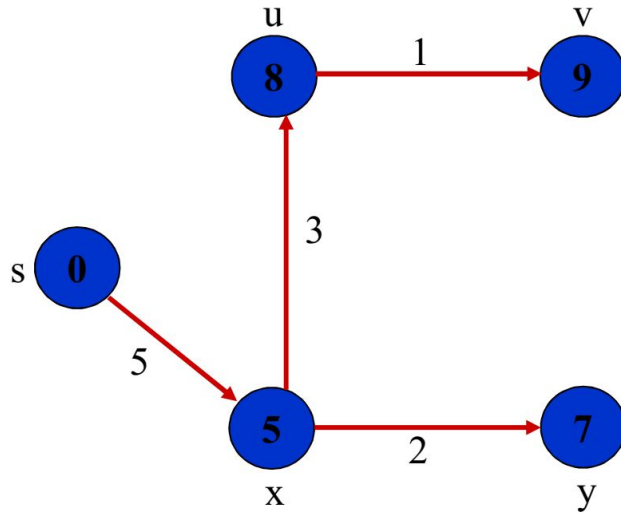
Dijkstra's Algorithm: Example



Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
14      if dist[u] + w(u, v) < dist[v]:
15        dist[v] ← dist[u] + w(u, v)
16        prev[v] ← u
17
18  return dist[], prev[]
```

Dijkstra's Algorithm: Example



Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
14      if dist[u] + w(u, v) < dist[v]:
15        dist[v] ← dist[u] + w(u, v)
16        prev[v] ← u
17
18
19  return dist[], prev[]
```

Dijkstra Algorithm Pseudocode

```
1 function Dijkstra(Graph, source):
2
3   for each vertex v in Graph.Vertices:
4     dist[v] ← INFINITY
5     prev[v] ← UNDEFINED
6     add v to Q
7   dist[source] ← 0
8
9   while Q is not empty:
10    u ← vertex in Q with min dist[u]
11    remove u from Q
12
13    for each unvisited neighbor v:
15      if dist[u] + w(u, v) < dist[v]:
16        dist[v] ← dist[u] + w(u, v)
17        prev[v] ← u
18
19  return dist[], prev[]
```

Relaxation

Very similar to BFS

- Instead of regular Queue, we use Min Heap

Dijkstra's Algorithm: Time Complexity

Using Array:

The simplest implementation of the Dijkstra's algorithm stores vertices in an ordinary array

- Good for dense graphs (many edges)
- $|V|$ vertices and $|E|$ edges
- Initialization $O(|V|)$
- While loop $O(|V|)$
 - Find and remove a min distance vertex $O(|V|)$
 - Potentially $|E|$ updates
 - ◆ Each update costs $O(1)$
- Reconstruct path $O(|E|)$

Total time $O(|V|^2 + |E|) = O(|V|^2)$

Dijkstra's Algorithm: Time Complexity

Using Priority Queue:

For sparse graphs, Dijkstra's Algorithm can be implemented more efficiently by *priority queue*

- Initialization $O(|V|)$ using $O(|V|)$ buildHeap()
- While loop $O(|V|)$
 - Find and remove a min dist vertex needs $O(\log |V|)$ using $O(\log |V|)$ ExtractMin()
 - Potentially $|E|$ updates
 - ◆ Each update costs $O(\log |V|)$ using decreaseKey()
- Reconstruct path $O(|E|)$

Total time $O(|V|\log|V| + |E|\log|V|) = O(|E|\log|V|)$

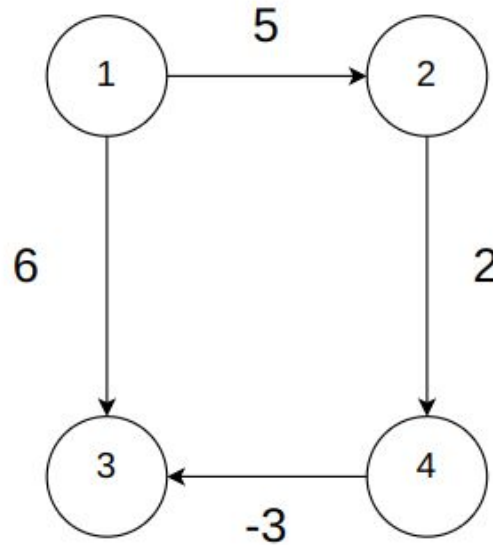
Example usage of Dijkstra's algorithm:

```
graph = {  
    'A': {'B': 1, 'C': 4},  
    'B': {'A': 1, 'C': 2, 'D': 5},  
    'C': {'A': 4, 'B': 2, 'D': 1},  
    'D': {'B': 5, 'C': 1}  
}  
  
start_vertex = 'A'  
result = dijkstra(graph, start_vertex)  
print(result)
```

Limitation of Dijkstra

- Might fail to find minimum distance in case of negative weights.

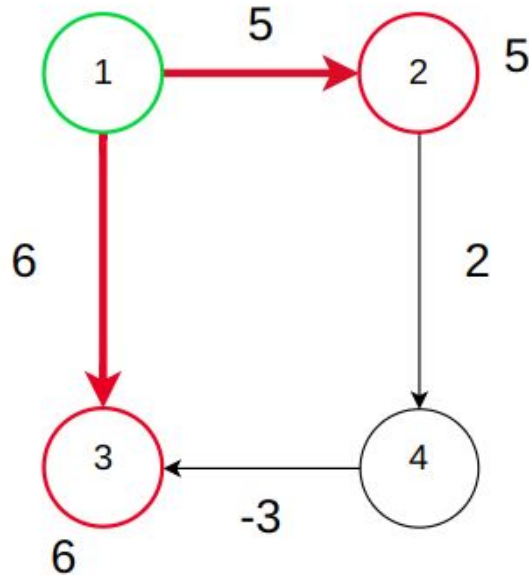
Example:



Limitation of Dijkstra

- Might fail to find minimum distance in case of negative weights.

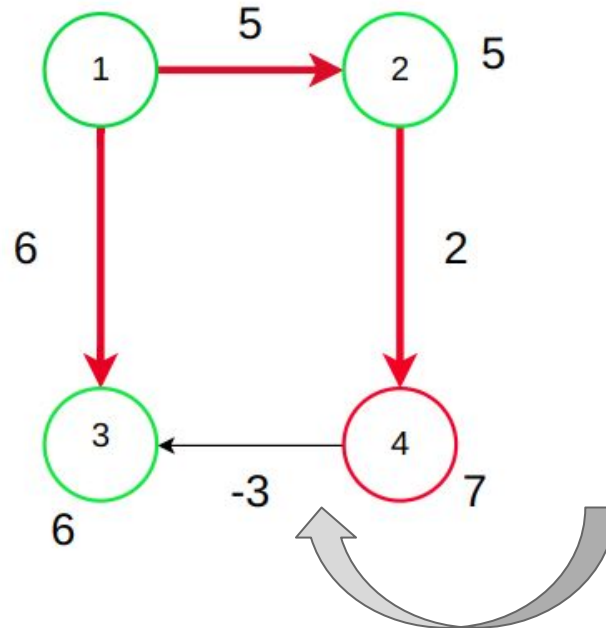
Example:



Limitation of Dijkstra

- Might fail to find minimum distance in case of negative weights.

Example:



Dijkstra will not relax this edge because vertex 3 is already visited. But if it is checked, the distance of vertex 3 should become $7 - 3 = 4$. So dijkstra cannot give correct answer in this case.

Limitation of Dijkstra

- Might fail to find minimum distance in case of negative weights.

So, Dijkstra may or may not give the correct result for negative edge weights.

Solution: Bellman Ford

Bellman-Ford Algorithm

BellmanFord()

```
1  for each  $v \in V$ 
2       $d[v] = \infty$ ;
3   $d[s] = 0$ ;

4  for  $i=1$  to  $|V|-1$ 
5      for each edge  $(u,v) \in E$ 
6          Relax( $u,v,w$ );

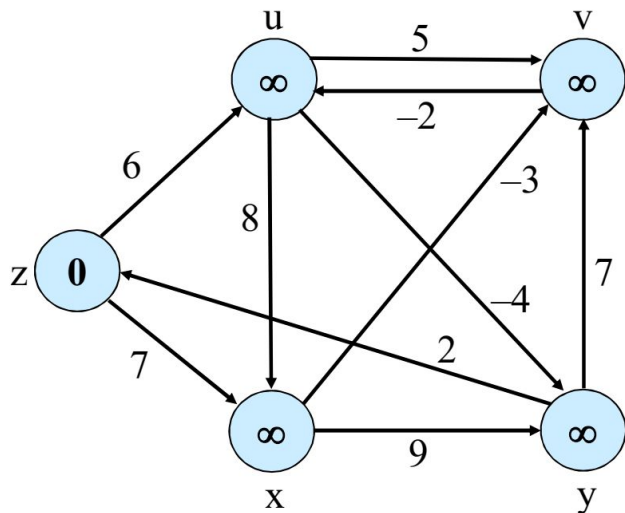
7  for each edge  $(u,v) \in E$ 
8      if ( $d[v] > d[u] + w(u,v)$ )
9          return "no solution";
```

Q: What will be the running time?

A: $O(VE)$

```
Relax( $u,v,w$ ): if ( $d[v] > d[u] + w(u,v)$ )
                then  $d[v] = d[u] + w(u,v)$ 
```

Bellman-Ford Algorithm: Example

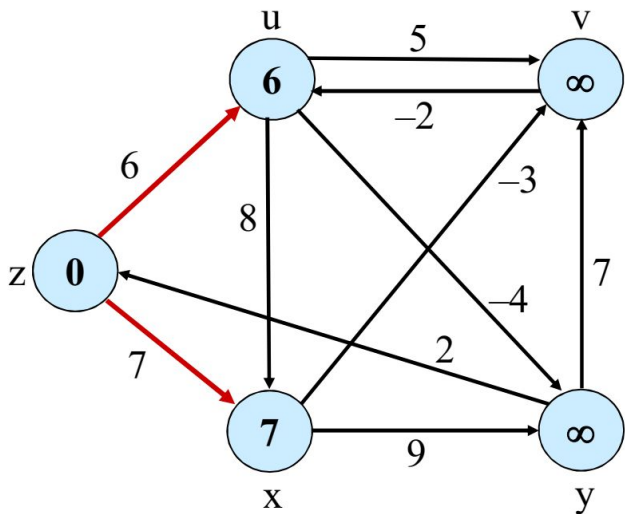


```
BellmanFord()
1  for each v ∈ V
2      d[v] = ∞;
3  d[s] = 0;

4  for i=1 to |V|-1
5      for each edge (u,v) ∈ E
6          Relax(u,v,w);

7  for each edge (u,v) ∈ E
8      if (d[v] > d[u] + w(u,v))
9          return "no solution";
```

Bellman-Ford Algorithm: Example

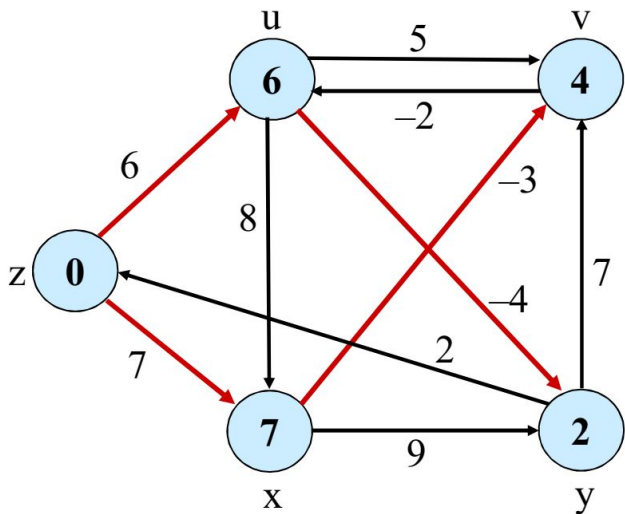


```
BellmanFord()
1  for each v  $\in$  V
2      d[v] =  $\infty$ ;
3  d[s] = 0;

4  for i=1 to |V|-1
5      for each edge (u,v)  $\in$  E
6          Relax(u,v,w);

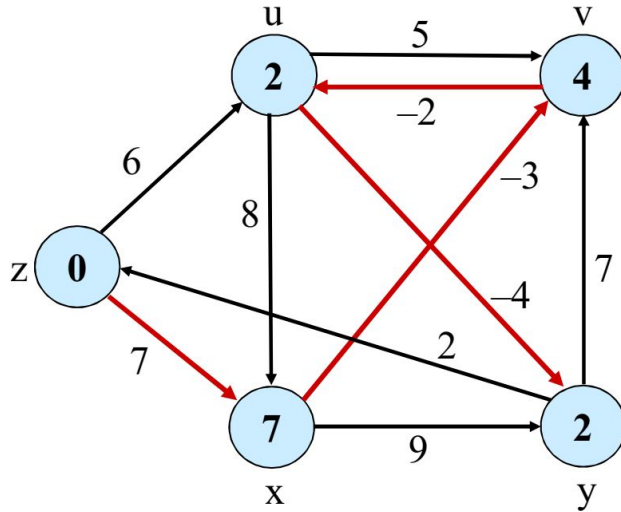
7  for each edge (u,v)  $\in$  E
8      if (d[v] > d[u] + w(u,v))
9          return "no solution";
```

Bellman-Ford Algorithm: Example



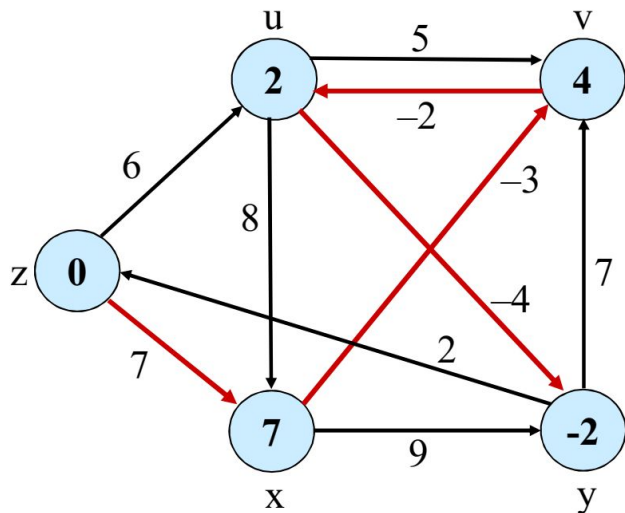
```
BellmanFord()  
1  for each v ∈ V  
2    d[v] = ∞;  
3  d[s] = 0;  
  
4  for i=1 to |V|-1  
5    for each edge (u,v) ∈ E  
6      Relax(u,v,w);  
  
7  for each edge (u,v) ∈ E  
8    if (d[v] > d[u] + w(u,v))  
9      return "no solution";
```

Bellman-Ford Algorithm: Example



```
BellmanFord()  
1  for each v ∈ V  
2    d[v] = ∞;  
3  d[s] = 0;  
  
4  for i=1 to |V|-1  
5    for each edge (u,v) ∈ E  
6      Relax(u,v,w);  
  
7  for each edge (u,v) ∈ E  
8    if (d[v] > d[u] + w(u,v))  
9      return "no solution";
```

Bellman-Ford Algorithm: Example



```
BellmanFord()
1  for each v ∈ V
2      d[v] = ∞;
3  d[s] = 0;

4  for i=1 to |V|-1
5      for each edge (u,v) ∈ E
6          Relax(u,v,w);

7  for each edge (u,v) ∈ E
8      if (d[v] > d[u] + w(u,v))
9          return "no solution";
```

Limitation of Bellman Ford Algorithm

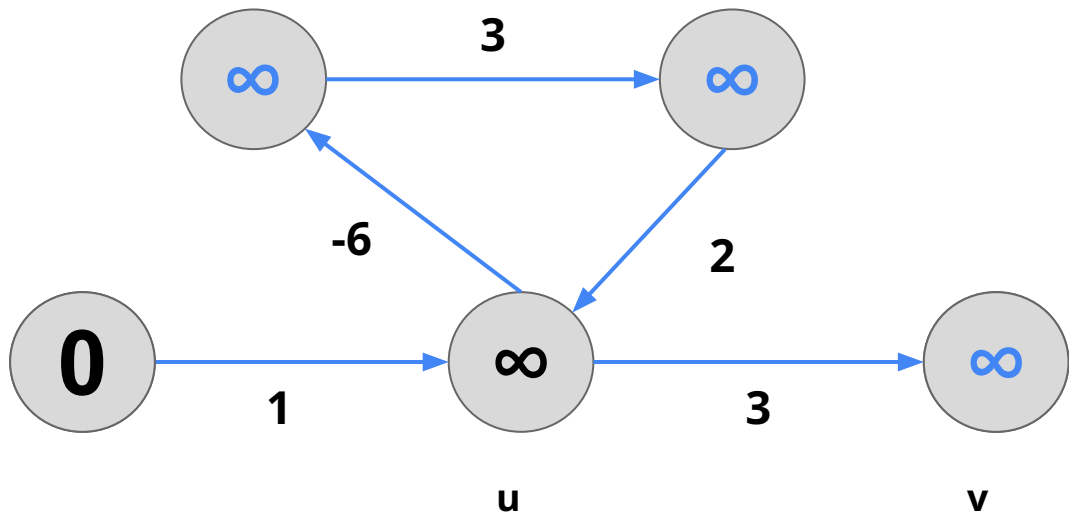
- Cannot find shortest path for negative weight cycle.
- After passing $|V| - 1$ times, there will still be some vertex, for which we did not yet reach the minimum distance. If we continue to run the algorithm again and again, it will keep decreasing. So, there will be no shortest path.

Limitation of Bellman Ford Algorithm

```
BellmanFord()  
1   for each  $v \in V$   
2        $d[v] = \infty$ ;  
3    $d[s] = 0$ ;  
  
4   for  $i=1$  to  $|V|-1$   
5       for each edge  $(u,v) \in E$   
6           Relax( $u,v,w$ ) ;  
  
7   for each edge  $(u,v) \in E$   
8       if  $(d[v] > d[u] + w(u,v))$   
9           return "no solution";
```



This additional checking finds out if there is any negative weight cycle and in this case, there will be no solution.



BellmanFord()

```

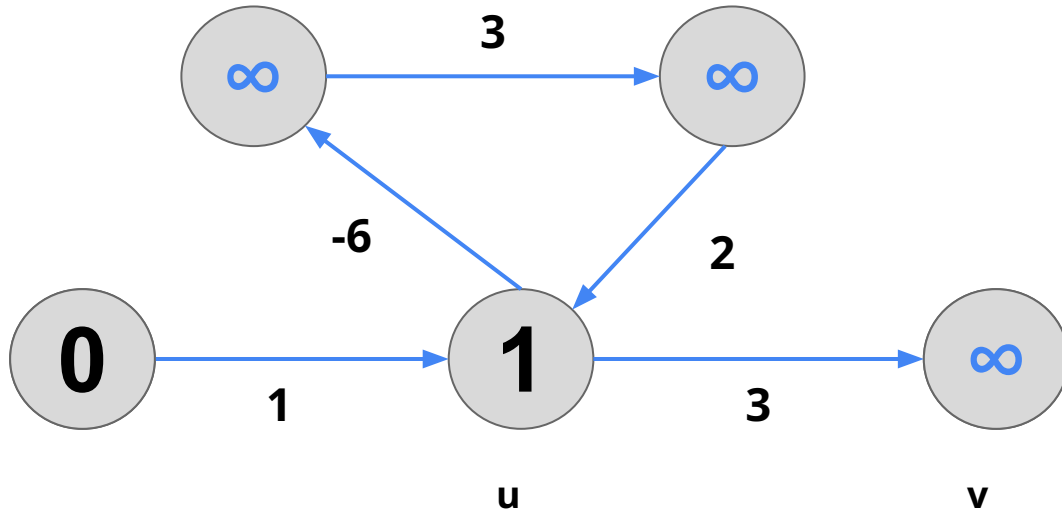
1  for each v ∈ V
2    d[v] = ∞;
3  d[s] = 0;

4  for i=1 to |V|-1
5    for each edge (u,v) ∈ E
6      Relax(u,v,w);

7  for each edge (u,v) ∈ E
8    if (d[v] > d[u] + w(u,v))
9      return "no solution";
  
```

1 pass complete out of 4 passes

$$|V|-1 = 5-1 = 4$$

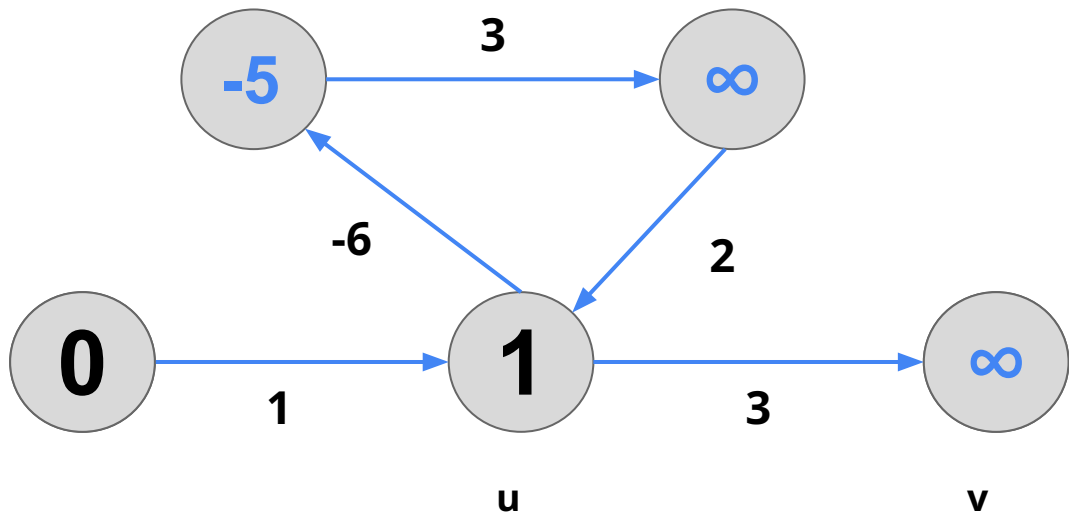


BellmanFord()

```
1  for each v ∈ V
2    d[v] = ∞;
3  d[s] = 0;

4  for i=1 to |V|-1
5    for each edge (u,v) ∈ E
6      Relax(u,v,w);

7  for each edge (u,v) ∈ E
8    if (d[v] > d[u] + w(u,v))
9      return "no solution";
```



```

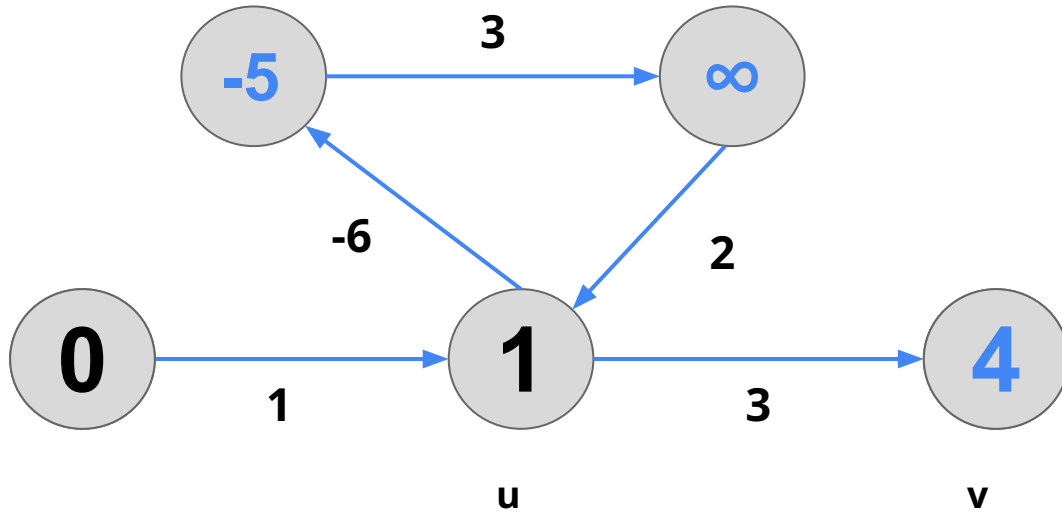
BellmanFord()
1  for each  $v \in V$ 
2     $d[v] = \infty$ ;
3   $d[s] = 0$ ;

4  for  $i=1$  to  $|V|-1$ 
5    for each edge  $(u,v) \in E$ 
6      Relax( $u,v,w$ ) ;

7  for each edge  $(u,v) \in E$ 
8    if ( $d[v] > d[u] + w(u,v)$ )
9      return "no solution";
  
```

2 passes complete out of 4 passes

$$|v| - 1 = 5 - 1 = 4$$



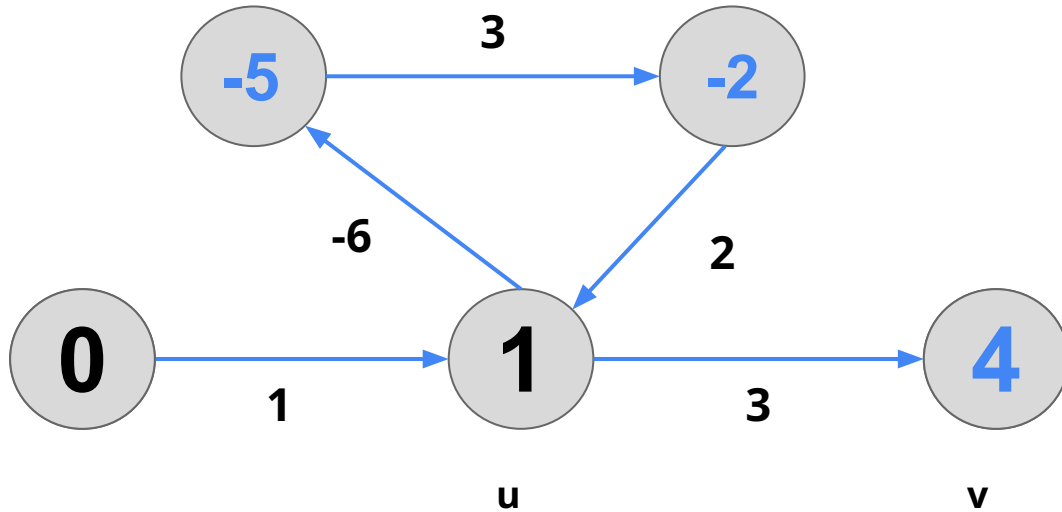
BellmanFord()

```
1  for each v ∈ V
2    d[v] = ∞;
3  d[s] = 0;

4  for i=1 to |V|-1
5    for each edge (u,v) ∈ E
6      Relax(u,v,w);

7  for each edge (u,v) ∈ E
8    if (d[v] > d[u] + w(u,v))
9      return "no solution";
```

3 passes complete out of 4 passes



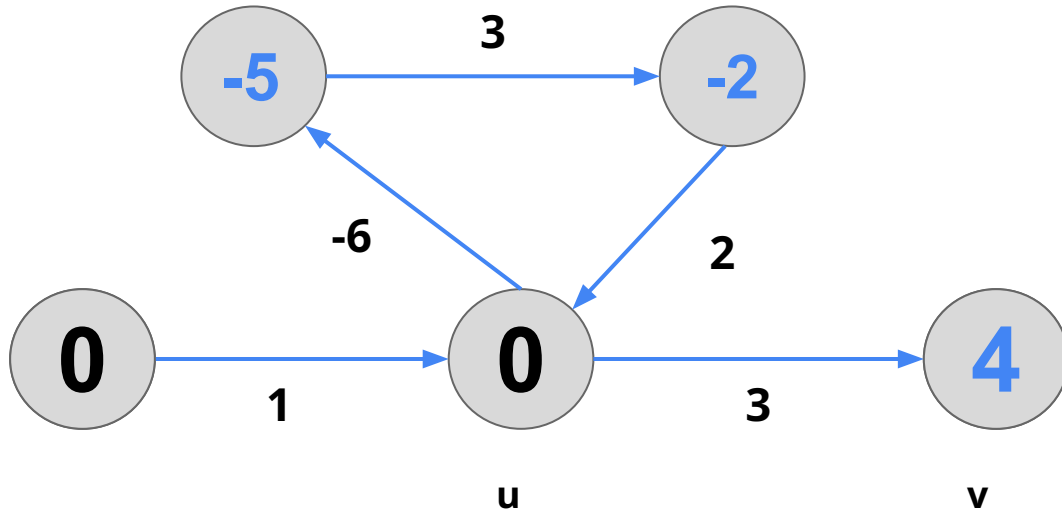
BellmanFord()

```
1  for each  $v \in V$ 
2     $d[v] = \infty$ ;
3   $d[s] = 0$ ;

4  for  $i=1$  to  $|V|-1$ 
5    for each edge  $(u,v) \in E$ 
6      Relax( $u,v,w$ ) ;

7  for each edge  $(u,v) \in E$ 
8    if  $(d[v] > d[u] + w(u,v))$ 
9      return "no solution";
```

4 passes complete out of 4 passes

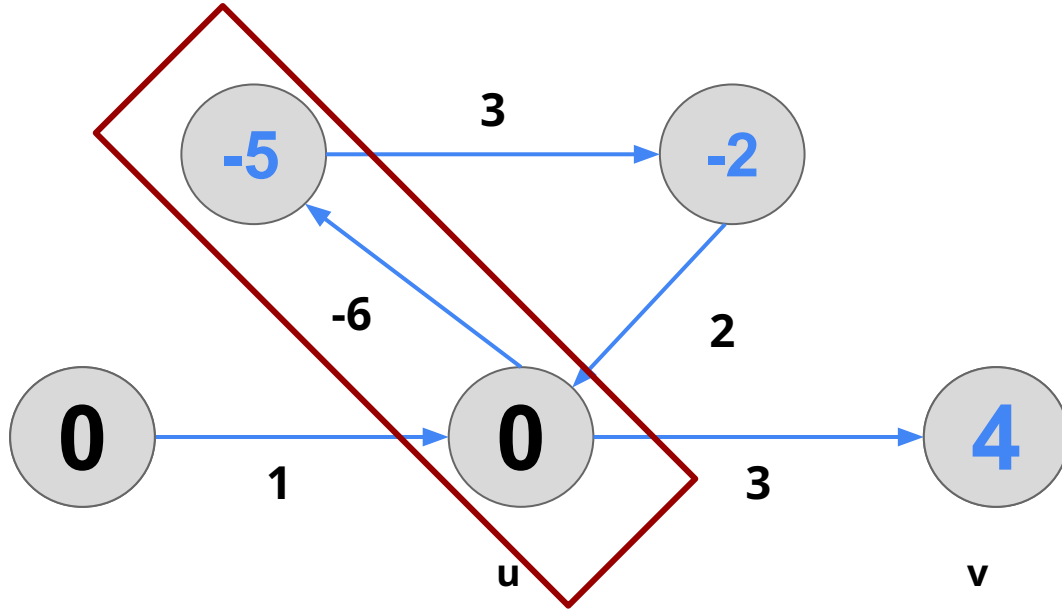


BellmanFord()

```
1  for each  $v \in V$ 
2     $d[v] = \infty$ ;
3   $d[s] = 0$ ;

4  for  $i=1$  to  $|V|-1$ 
5    for each edge  $(u,v) \in E$ 
6      Relax( $u,v,w$ ) ;

7  for each edge  $(u,v) \in E$ 
8    if  $(d[v] > d[u] + w(u,v))$ 
9      return "no solution";
```



BellmanFord()

```

1  for each v ∈ V
2    d[v] = ∞;
3  d[s] = 0;

4  for i=1 to |V|-1
5    for each edge (u,v) ∈ E
6      Relax(u,v,w);

7  for each edge (u,v) ∈ E
8    if (d[v] > d[u] + w(u,v))
9      return "no solution";
  
```

$$0 - 6 = -6 < -5$$

Correctness of Bellman-Ford Algorithm

- After $|V|-1$ passes, all d values are correct
 - Consider shortest path from s to v :
 $s \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_k = v$, where $k \leq |V|-1$
 - ◆ Initially, $d[s] = 0$ is correct, and doesn't change
 - ◆ After 1 pass through edges, $d[v_1]$ is correct and doesn't change
 - ◆ After 2 passes, $d[v_2]$ is correct and doesn't change
 - ◆ ...
 - ◆ Terminates in $|V| - 1$ passes.

Time Complexity of Bellman Ford

→ Relax $|E|$ edges for $|V| - 1$ times. So time complexity is $O(|E| * (|V| - 1)) = O(|E||V|) = O(n^2)$.

→ If the graph is dense that means, if there are edges between all pairs of vertices, then $|E| = n * (n - 1) / 2$. SO the time complexity becomes $O(n^3)$.

Thus Bellman-Ford takes more time than dijkstra but it ensures the correct result.

Example usage of Bellman-Ford algorithm:

```
graph = {  
    'A': {'B': 1, 'C': 4},  
    'B': {'C': 2, 'D': 5},  
    'C': {'D': 1},  
    'D': {'B': -10}  
}
```

```
start_vertex = 'A'  
result = bellman_ford(graph, start_vertex)  
print(result)
```

Resources

- [3.6 Dijkstra Algorithm - Single Source Shortest Path - Greedy Method - YouTube](#)
- [4.4 Bellman Ford Algorithm - Single Source Shortest Path - Dynamic Programming - YouTube](#)